

LAB-01 (A)

Problem Statement:- The task is to design a 16-bit ALU (Arithmetic Logic Unit) and create a self-checking SystemVerilog testbench to verify its correctness whether it is showing overflow or not.

Verification plan:-

Verification plan.

Multi-stage pipelined ALU

1. objective

To design & verify a multi-stage pipelined ALU using System verilog.

2. Design under Test (DUT)

The DUT is a pipelined ALU with multiple pipeline stages.

3. Pipeline Stages

1. Input register
2. ALU Computation.
3. output register.

4. Test Scenarios / Test Cases

5. self-checking mechanism.

If DUT-result == expected-result $\rightarrow$ pass
else $\rightarrow$ FAIL

6. Corner Test Cases

- Reset during pipeline operation
- Valid in = 0 for multiple cycles
- Maximum operands
- minimum minimum operands

7. pass Criteria.

Design is Verified

- All operations produce correct result

Code:-

```
// Code your design here
module ALU16 (
    input [15:0] A,    // Operand A
    input [15:0] B,    // Operand B
    input [3:0] ALU_Sel, // Operation select
    output reg [15:0] ALU_Out,
    output reg CarryOut,
    output Zero
);

    wire [16:0] tmp_add; // For addition with carry
    wire [16:0] tmp_sub; // For subtraction with borrow

    assign tmp_add = {1'b0, A} + {1'b0, B};
    assign tmp_sub = {1'b0, A} - {1'b0, B};

    // Zero flag
    assign Zero = (ALU_Out == 16'b0);

    always @(*) begin
        case(ALU_Sel)

            4'b0000: begin // Addition
                ALU_Out = tmp_add[15:0];
                CarryOut = tmp_add[16];
            end

            4'b0001: begin // Subtraction
                ALU_Out = tmp_sub[15:0];
                CarryOut = tmp_sub[16]; // borrow flag
            end

            4'b0010: begin // AND
                ALU_Out = A & B;
                CarryOut = 0;
            end

            4'b0011: begin // OR
                ALU_Out = A | B;
                CarryOut = 0;
            end

            4'b0100: begin // XOR
```

```

        ALU_Out = A ^ B;
        CarryOut = 0;
    end

    4'b0101: begin // NOT (A)
        ALU_Out = ~A;
        CarryOut = 0;
    end

    4'b0110: begin // Logical Shift Left
        ALU_Out = A << 1;
        CarryOut = A[15];
    end

    4'b0111: begin // Logical Shift Right
        ALU_Out = A >> 1;
        CarryOut = A[0];
    end

    4'b1000: begin // Arithmetic Shift Right
        ALU_Out = $signed(A) >>> 1;
        CarryOut = A[0];
    end

    4'b1001: begin // Compare (A == B)
        ALU_Out = (A == B) ? 16'd1 : 16'd0;
        CarryOut = 0;
    end

    4'b1010: begin // Compare (A > B)
        ALU_Out = (A > B) ? 16'd1 : 16'd0;
        CarryOut = 0;
    end

    4'b1011: begin // Compare (A < B)
        ALU_Out = (A < B) ? 16'd1 : 16'd0;
        CarryOut = 0;
    end

    default: begin
        ALU_Out = 16'd0;
        CarryOut = 0;
    end
endcase
end

endmodule

```

Unsigned overflow checker Test bench:-

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module ALU16_tb;

    // VCD + EPWave
    initial begin
        $dumpfile("alu16.vcd"); // VCD file for EPWave
        $dumpvars(0, ALU16_tb); // Dump all signals in testbench + DUT
    end

    // DUT inputs
    reg [15:0] A, B;
    reg [3:0] ALU_Sel;

    // DUT outputs
    wire [15:0] ALU_Out;
    wire CarryOut;
    wire Zero;

    // Expected (Golden Reference)
    reg [15:0] exp_out;
    reg exp_carry;
    reg exp_zero;

    // Instantiate ALU (DUT)
    ALU16 dut (
        .A(A),
        .B(B),
        .ALU_Sel(ALU_Sel),
        .ALU_Out(ALU_Out),
        .CarryOut(CarryOut),
        .Zero(Zero)
    );

    // -----
    // Golden Reference Model
    // -----
    task golden_model;
        input [15:0] A, B;
        input [3:0] sel;

        begin
            case (sel)
```

```

4'b0000: begin // ADD
    {exp_carry, exp_out} = A + B;
end

4'b0001: begin // SUB
    {exp_carry, exp_out} = A - B;
end

4'b0010: begin // AND
    exp_out = A & B;
    exp_carry = 0;
end

4'b0011: begin // OR
    exp_out = A | B;
    exp_carry = 0;
end

4'b0100: begin // XOR
    exp_out = A ^ B;
    exp_carry = 0;
end

4'b0101: begin // NOT
    exp_out = ~A;
    exp_carry = 0;
end

4'b0110: begin // SHL
    exp_out = A << 1;
    exp_carry = A[15];
end

4'b0111: begin // SHR
    exp_out = A >> 1;
    exp_carry = A[0];
end

4'b1000: begin // Arithmetic Right Shift
    exp_out = $signed(A) >>> 1;
    exp_carry = A[0];
end

4'b1001: exp_out = (A == B); // EQ compare
4'b1010: exp_out = (A > B); // GT compare
4'b1011: exp_out = (A < B); // LT compare

default: begin
    exp_out = 0;
    exp_carry = 0;
end

```

```

        end
    endcase

    exp_zero = (exp_out == 0);
end

endtask

// -----
// Checker
// -----
task check_output;
begin
    if (ALU_Out === exp_out && CarryOut === exp_carry && Zero === exp_zero)
        $display("PASS: sel=%b A=%h B=%h OUT=%h", ALU_Sel, A, B, ALU_Out);
    else begin
        $display("FAIL: sel=%b A=%h B=%h", ALU_Sel, A, B);
        $display(" Expected: OUT=%h Carry=%b Zero=%b", exp_out, exp_carry,
exp_zero);
        $display(" Got:    OUT=%h Carry=%b Zero=%b", ALU_Out, CarryOut, Zero);
    end
end
endtask

// -----
// Testcases
// -----
initial begin
    $display("----- SELF CHECKING ALU TESTBENCH STARTED -----");

    repeat (20) begin
        // Random inputs
        A = $random;
        B = $random;
        ALU_Sel = $random % 12;

        #5;

        // Call golden model
        golden_model(A, B, ALU_Sel);

        // Compare DUT vs golden
        check_output();

        #5;
    end

    $display("----- TEST COMPLETE -----");
    $finish;
end

```

```
endmodule
```

Signed Testbench Checker:-

```
`timescale 1ns/1ps

module ALU16_tb;

    // VCD for EPWave
    initial begin
        $dumpfile("alu16.vcd");
        $dumpvars(0, ALU16_tb);
    end

    // DUT inputs
    reg [15:0] A, B;
    reg [3:0] ALU_Sel;

    // DUT outputs
    wire [15:0] ALU_Out;
    wire CarryOut;
    wire Zero;

    // Expected outputs
    reg [15:0] exp_out;
    reg exp_carry;
    reg exp_zero;

    // <ADD> Expected signed overflow
    reg exp_signed_overflow;

    // DUT instance
    ALU16 dut (
        .A(A),
        .B(B),
        .ALU_Sel(ALU_Sel),
        .ALU_Out(ALU_Out),
        .CarryOut(CarryOut),
        .Zero(Zero)
    );

    // -----
    // Golden Reference Model
    // -----
    task golden_model;
        input [15:0] A, B;
```



```

input [3:0] sel;

begin
  case (sel)

    4'b0000: begin // ADD
      {exp_carry, exp_out} = A + B;

      // ADD signed overflow
      exp_signed_overflow =
        (A[15] == B[15]) && (exp_out[15] != A[15]);
    end

    4'b0001: begin // SUB
      {exp_carry, exp_out} = A - B;

      // SUB signed overflow
      exp_signed_overflow =
        (A[15] != B[15]) && (exp_out[15] != A[15]);
    end

    default: begin
      exp_out = 0;
      exp_carry = 0;
      exp_signed_overflow = 0;
    end
  endcase

  exp_zero = (exp_out == 0);
end
endtask

// -----
// Checker
// -----
task check_output;
begin
  // FAIL if signed overflow is detected
  if (exp_signed_overflow) begin
    $display("FAIL (SIGNED OVERFLOW): sel=%b A=%h B=%h OUT=%h",
      ALU_Sel, A, B, exp_out);
  end

  // PASS if everything matches
  else if (ALU_Out === exp_out &&
    CarryOut === exp_carry &&
    Zero === exp_zero) begin

    $display("PASS: sel=%b A=%h B=%h OUT=%h",
      ALU_Sel, A, B, ALU_Out);
  end
end

```

```

// FAIL if output mismatch
else begin
    $display("FAIL: sel=%b A=%h B=%h", ALU_Sel, A, B);
    $display("Expected: OUT=%h Carry=%b Zero=%b",
        exp_out, exp_carry, exp_zero);
    $display("Got:    OUT=%h Carry=%b Zero=%b",
        ALU_Out, CarryOut, Zero);
end
end
endtask

// -----
// Testcases
// -----
initial begin
    $display("----- SELF CHECKING ALU TESTBENCH STARTED -----");

    repeat (20) begin
        A = $random;
        B = $random;
        ALU_Sel = $random % 12;

        #5;
        golden_model(A, B, ALU_Sel);
        check_output();
        #5;
    end

    $display("----- TEST COMPLETE -----");
    $finish;
end

endmodule

```

Output:-

LogShare

[2026-01-27 03:25:45 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv && unbuffer vvp a.out

warning: Some design elements have no explicit time unit and/or
: time precision. This may cause confusing timing results.
: Affected design elements are:
: -- module ALU16 declared here: design.sv:2

VCD info: dumpfile alu16.vcd opened for output.

----- SELF CHECKING ALU TESTBENCH STARTED -----

PASS: sel=1101 A=3524 B=5e81 OUT=0000

FAIL: sel=0101 A=5663 B=7b0d

Expected: OUT=0000 Carry=0 Zero=1

Got: OUT=a99c Carry=0 Zero=0

PASS: sel=1001 A=8465 B=5212 OUT=0000

FAIL: sel=0101 A=cd0d B=f176

Expected: OUT=0000 Carry=0 Zero=1

Got: OUT=32f2 Carry=0 Zero=0

PASS: sel=0001 A=57ed B=f78c OUT=6061

FAIL: sel=0110 A=24c6 B=84c5

Expected: OUT=0000 Carry=0 Zero=1

Got: OUT=498c Carry=0 Zero=0

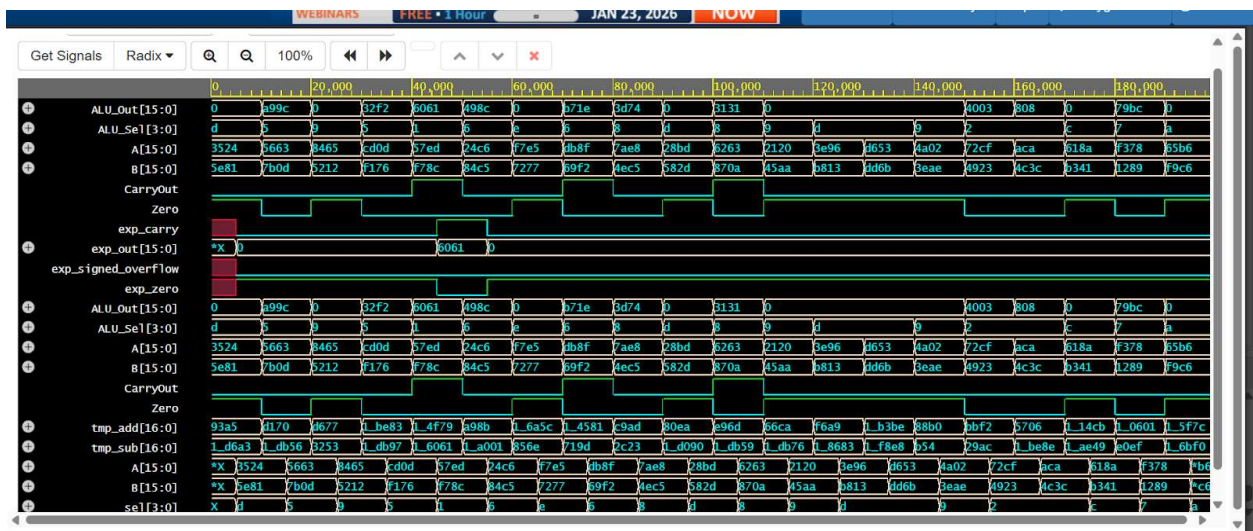
PASS: sel=1110 A=f7e5 B=7277 OUT=0000

FAIL: sel=0110 A=db8f B=69f2

Expected: OUT=0000 Carry=0 Zero=1

Got: OUT=b71e Carry=1 Zero=0

Waveforms:-



Explanation:-

I learned how a 16-bit ALU works and how different operations like add, subtract, shifts, and logic functions are performed.

I understood the concept of signed overflow and how to detect it correctly using sign bits.

I learned to write a golden reference model that calculates the expected output for verification.

In a 16-bit ALU, adding two numbers may produce a 17-bit result.

The ALU cannot store the 17th bit, so it becomes the CarryOut signal.

If CarryOut = 1, we call it unsigned overflow.

For signed overflow, we ignore the carry and check whether the sign bit flipped incorrectly.

If two positive numbers produce a negative sign, or two negative numbers produce a positive sign, that is signed overflow.

LAB 1(B)

Aim:- To design a self-checking testbench to verify the functionality of a 16-bit ripple carry adder.

Verification plan:-

Verification plan.

16 bit RCA

Step 1: Instantiate DUT

- 16 bit RCA
- Connect input A, B, Cin
- output Sum, Cout

Step 2: Declare file Handles

open input file $\rightarrow$ abc.txt

open output file $\rightarrow$ xyz.txt

Step 3: Declare Variables

Step 4: Read file line by line

Step 5: Apply Inputs to DUT

Step 6: Capture DUT output

Step 7: Compare Expected vs DUT

Step 8: Write Results to output File.

Step 9: Repeat for All Vectors.

Step 10: Close files.

Code

```
// Code your design here
// -----
// Full Adder Module (1-bit)
// -----
module full_adder(
    input A,
    input B,
```

```

    input Cin,    // Carry in
    output Sum,
    output Cout   // Carry out
);
    assign Sum = A ^ B ^ Cin;           // XOR for sum
    assign Cout = (A & B) | (Cin & (A ^ B)); // Carry logic
endmodule

// -----
// 16-bit Ripple Carry Adder Module
// -----
module rca_16bit(
    input [15:0] A,
    input [15:0] B,
    input      Cin,
    output [15:0] Sum,
    output      Cout
);

    wire [15:0] carry; // Internal carries

    // Connect first full adder
    full_adder FA0 (A[0], B[0], Cin, Sum[0], carry[0]);

    // Use a generate loop for remaining 15 adders
    genvar i;
    generate
        for(i = 1; i < 16; i = i + 1) begin
            full_adder FA (A[i], B[i], carry[i-1], Sum[i], carry[i]);
        end
    endgenerate

    assign Cout = carry[15]; // Final carry
endmodule

```

TestBench

```

`timescale 1ns/1ps

module rca_16bit_tb;

    reg [15:0] A, B;

```

```

reg      Cin;
wire [15:0] Sum;
wire      Cout;

reg [16:0] expected; // 17-bit reference (Sum + Cout)
integer file, scan;

integer pass_count = 0;
integer fail_count = 0;

// DUT
rca_16bit DUT (
    .A(A),
    .B(B),
    .Cin(Cin),
    .Sum(Sum),
    .Cout(Cout)
);

initial begin
    $dumpfile("rca16.vcd");
    $dumpvars(0, rca_16bit_tb);

    file = $fopen("inputs.txt", "r");
    if (file == 0) begin
        $display("Error: Could not open file!");
        $finish;
    end

    $display("A   B   Cin | Sum   Cout | Expected | Result");
    $display("-----");

    while (!$feof(file)) begin
        scan = $fscanf(file, "%h %h %d\n", A, B, Cin);
        #10;

        // Golden reference calculation
        expected = A + B + Cin;

        if ((Sum == expected[15:0]) && (Cout == expected[16])) begin
            $display("%04h %04h %0d | %04h %0d | %04h_%0d PASS",
                A, B, Cin, Sum, Cout, expected[15:0], expected[16]);
            pass_count = pass_count + 1;
        end
        else begin
            $display("%04h %04h %0d | %04h %0d | %04h_%0d FAIL",
                A, B, Cin, Sum, Cout, expected[15:0], expected[16]);
            fail_count = fail_count + 1;
        end
    end
end
end

```

```

    $display("\nTOTAL PASSED = %0d", pass_count);
    $display("TOTAL FAILED = %0d", fail_count);

    $fclose(file);
    $finish;
end

endmodule

```

Explanation:-

Output:-

The screenshot shows a simulation window with a warning message and a testbench result table. The warning states: "warning: Some design elements have no explicit time unit and/or : time precision. This may cause confusing timing results. : Affected design elements are: : -- module full_adder declared here: design.sv:5 : -- module rca_16bit declared here: design.sv:20". Below the warning, it says "VCD info: dumpfile rca16.vcd opened for output." and then displays a table of test results.

A	B	Cin	Sum	Cout	Expected	Result
f6e7	8f87	0	866e	1	866e_1	PASS
7c6d	558b	1	d1f9	0	d1f9_0	PASS
f598	1786	1	0d1f	1	0d1f_1	PASS
81dc	311e	1	b2fb	0	b2fb_0	PASS
dbec	c727	1	a314	1	a314_1	PASS
6b88	3fa9	1	ab32	0	ab32_0	PASS
119d	1bd5	1	2d73	0	2d73_0	PASS
9942	05af	0	9ef1	0	9ef1_0	PASS
651c	f866	0	5d82	1	5d82_1	PASS
3e9b	8d3e	0	cbd9	0	cbd9_0	PASS
5bb4	ed38	1	48ed	1	48ed_1	PASS